Review and Implementation of Permutation Sampling Algorithms

Isfandiyor Mullajonov

September 17, 2025

Acknowledgement

The author thanks Dr. Jean Peyen from DIICSU for contributing to this research paper as a mentor for the Lumiere research program.

1 Introduction

Permutation sampling, also known as random shuffling, refers to the process of rearranging a collection's elements into a uniformly random order. At its core, the study of permutation sampling relies on combinatorics, probability theory, and algorithmic analysis, as these tools are essential both to establish the uniformity of the shuffle and to evaluate its computational efficiency. Beyond the conceptual appeal, it plays a central role in a variety of computational and mathematical contexts, such as in generating instances to benchmark sorting algorithms, statistics, and cryptography. The aim of this paper is to perform a systematic comparative analysis of shuffling algorithms by reviewing existing algorithms, exploring new implementations, and presenting benchmark results with analysis.

2 The Fisher–Yates algorithm

The Fisher-Yates algorithm [2], also known as the Knuth shuffle, is an algorithm for randomly shuffling a finite sequence. It ensures that every possible permutation of the sequence is equally likely to occur provided that all the elements of the sequence are distinct. This means that it's fair and unbiased shuffle, in other words, it is a uniform sampler. It works by iterating through the sequence and swapping each element with a randomly chosen element from the unshuffled portion of the sequence.

Algorithm 1 Fisher–Yates

Require: Array A of length $n \geq 2$
Ensure: Random permutation of A

```
for i ← n downto 1 do
    j ← random integer such that 1 ≤ j ≤ i
    Swap A[i] and A[j]
end for
return A
```

Theoretical justification: At each step i from n to 1, the algorithm chooses a random index j in $[1, i]$ and swaps $A[i]$ with $A[j]$. This results in $n \cdot (n-1) \cdots 2 = n!$ unique execution paths, each corresponding to a distinct permutation. Because the choices are mutually independent, every permutation is generated with probability $1/n!$. Hence, the algorithm produces a uniform random permutation.

A Python implementation of Fisher-Yates algorithm can be found in [4].

This implementation is tested against the Numpy implementation [4]. Figure 1, compares their time-execution on a MacBook Air M2, which includes an 8-core CPU (4 high-performance cores at up to 3.49 GHz, 4 efficiency cores at up to 2.42 GHz). The same device is used in latter benchmarks.

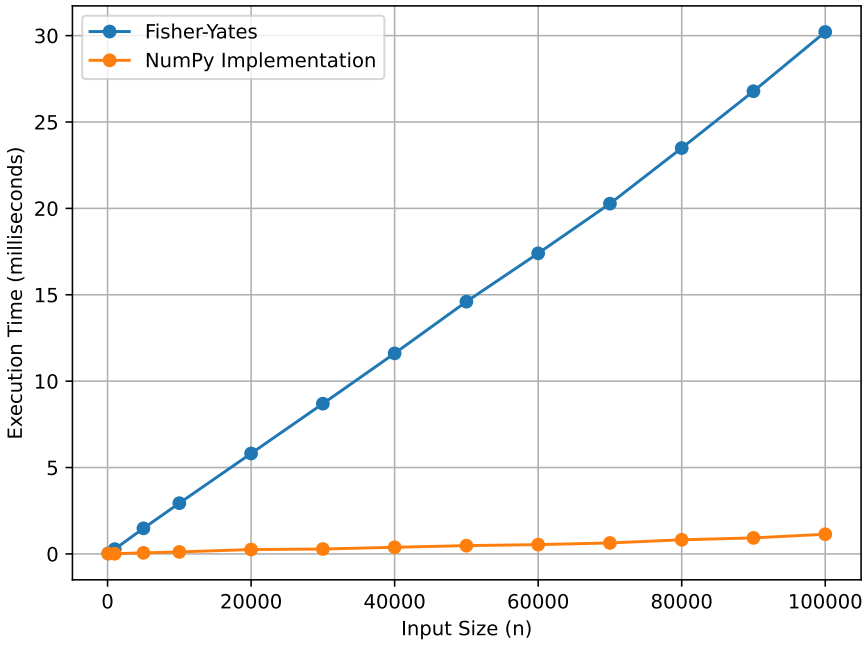


Figure 1: Permutation sampling Execution-time Comparison

As expected, the Fisher–Yates algorithm exhibits approximately linear time complexity, with the regression result:

$$Time_{Fisher-Yates} \approx 0.0003 \cdot n + 0.12 \; [ms]$$

On the other hand, the NumPy implementation shows significantly better performance, with the regression yielding:

$$Time_{NumPy} \approx 0.000007 \cdot n + 0.03 \; [ms]$$

This can be attributed to NumPy's use of precompiled C code and internal optimizations.

3 Randomized Key Sorting Algorithm

In this section, two parallel sampler algorithms were implemented for generating random permutations based on uniform distribution [4]. A parallel sampler refers to an algorithm designed to perform sampling tasks, such as random number generation and permutation construction, across multiple processing units simultaneously [3].

3.1 Uniform-Sort Based Implementation

Algorithm 2 Uniform-Sort Based Algorithm

Require: n ▷ Size of permutation
Ensure: π ▷ Random permutation
1: **for all** $i = 1$ to n **in parallel do**
2: $r[i] \leftarrow \text{Uniform}(0, 1)$
3: **end for**
4: $sorted_indices \leftarrow \texttt{argsort}(r)$
5: $\pi \leftarrow sorted_indices$
6: **return** π

The approach involves assigning each element a uniformly sampled (pseudo-)random number and then sorting the resulting values to determine the permutation order. It is particularly suitable for parallel execution, as random number generation and sorting can be distributed across processing units. The algorithm makes use of *argsort*, which by default applies the quicksort algorithm, and *uniform* functions from the NumPy library [5]. This method has been evaluated over multiple runs at varying input sizes to analyze the variability in execution time. In Figure 3, visualization of the results using mean times with standard deviation and in Figure 2, mean execution time split can be seen.

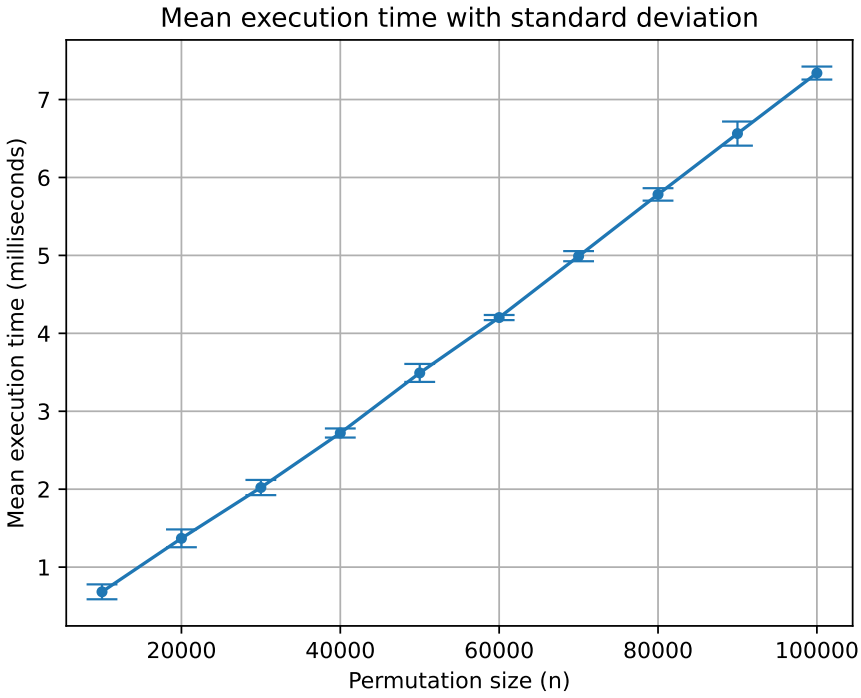


Figure 2: Execution-time of an implementation of Algorithm 2

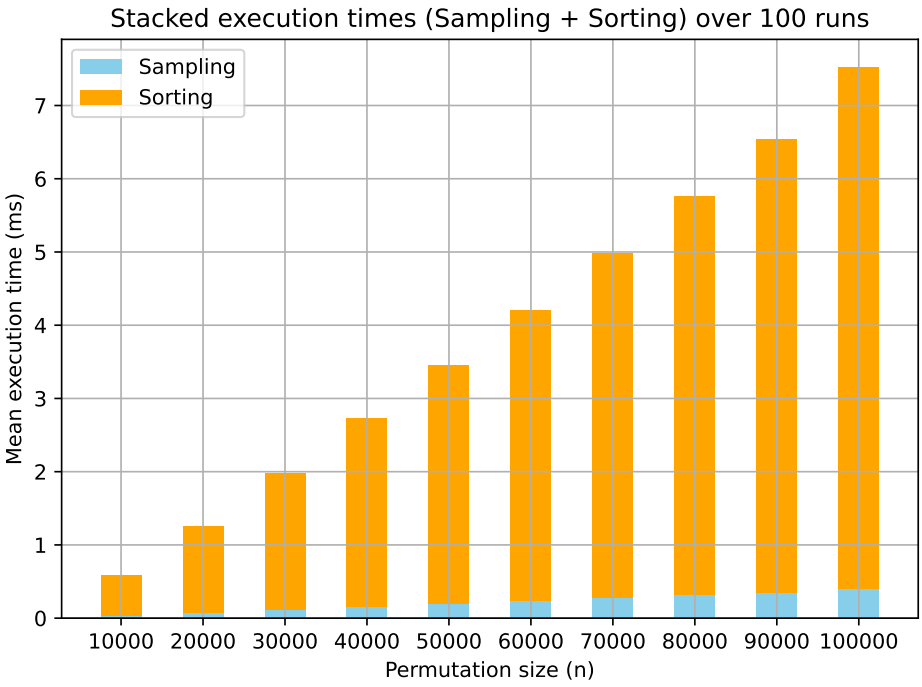


Figure 3: Execution time split

Figure 3 illustrates the average execution time of the permutation sampler broken down into two distinct phases: uniform random number generation (sampling) and sorting. The results, averaged over 100 independent runs for each permutation size, show that sorting time increases significantly faster than sampling time as the input size grows, and it accounts for most of the total execution time.

3.2 Thread-Based Parallelization

Algorithm 3 Parallelized Algorithm Using Threads

Require: n ▷ Size of permutation
Require: T ▷ Number of threads
Ensure: π ▷ Random permutation
1: Generate array $r[1 \dots n]$ with random values from Uniform$(0, 1)$
2: Generate array $b[1 \dots n]$ with random integers from 0 to $T - 1$ ▷ Thread buckets
3: **for all** $t = 0$ to $T - 1$ **in parallel do**
4: Let $B_t \leftarrow \{i \mid b[i] = t\}$ ▷ Indices for thread t
5: Sort B_t using values $r[i]$ to get sorted indices S_t
6: **end for**
7: Concatenate all $S_0, S_1, \dots, S_{T-1}$ in order to get a permutation π
8: **return** π

This version of the sorting algorithm incorporates multithreading to execute parts of the algorithm concurrently. Specifically, elements are randomly assigned to thread-specific buckets, and each bucket is sorted independently using local random keys. We use Python's *ThreadPoolExecutor* to parallelize this process, allowing multiple threads to perform sorting simultaneously. [4]

From the benchmark in Figure 4, it is evident that the multi-threaded implementation of the algorithm gives an improvement in execution time relative to the baseline *argsort*-based version. The figure reports results for a configuration utilizing four threads; however, this parameter can be adapted to align with the processing capabilities of the user's CPU. The performance of the multi-threaded algorithm approaches that of NumPy's optimized shuffle routine, benefiting from increased parallelism while maintaining scalability across varying thread counts.

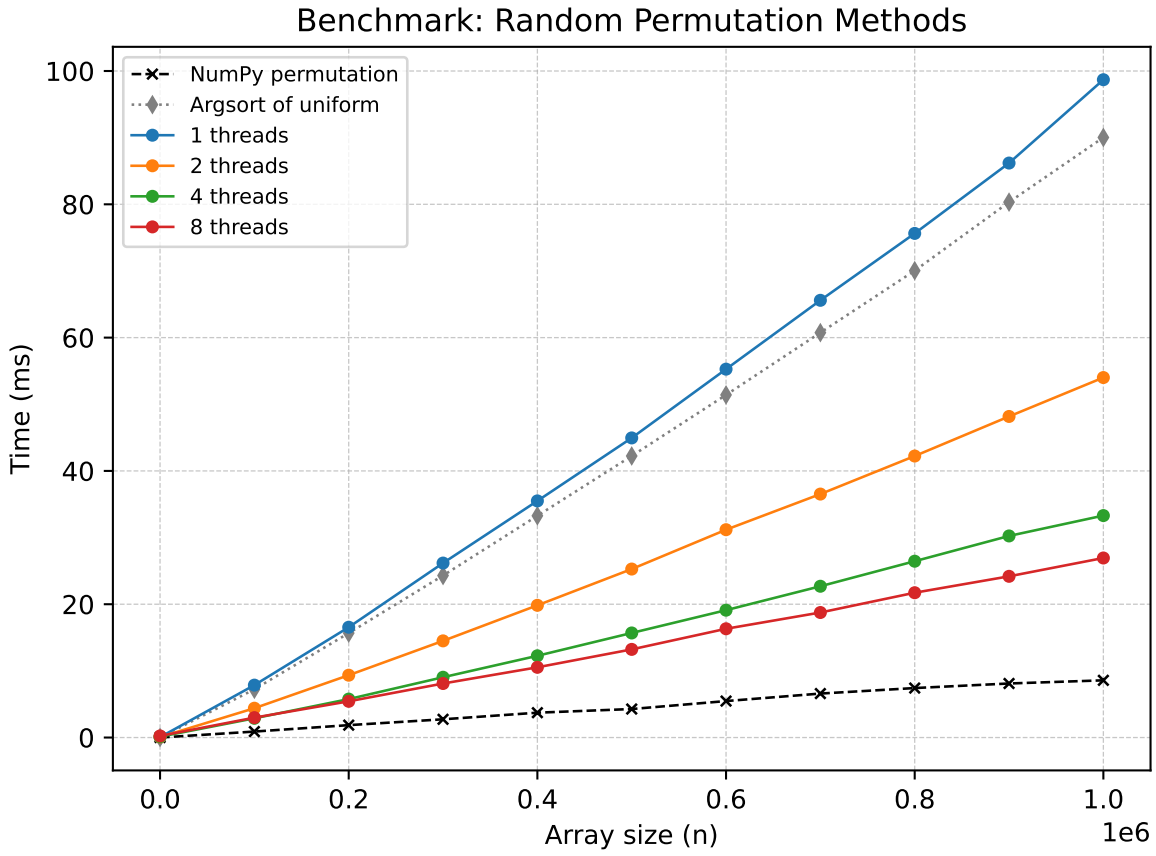


Figure 4: Execution-time comparison

4 Review of sorting algorithms

In the previous section, we established that random shuffling can be viewed as conceptually equivalent to assigning each element an independent, identically distributed random key, theoretically drawn from an absolutely continuous distribution, and then sorting the elements based on these keys. This equivalence between shuffling and sorting highlights the need for the review of sorting algorithms. Therefore, two classical sorting algorithms which are mergesort and quicksort will be discussed in detail.

4.1 Merge Sort

Merge Sort is a divide-and-conquer algorithm that recursively splits the input list into two halves, sorts each half, and merges them into a fully sorted list. It is stable, guarantees $O(n \log n)$ performance even in the worst case, and is often favored for linked lists and parallel applications due to its predictable and balanced recursion structure. Its memory requirement is higher than in-place algorithms since it requires auxiliary arrays

for merging. Merge Sort consistently partitions the input in half, resulting in reliable execution times, and is better suited for scenarios where deterministic performance and parallelism are preferred. Although non-recursive versions exist, they rarely offer significant speedups over the recursive version. [1, 6]

Algorithm 4 MergeSort

Require: array
Ensure: sorted array

```
1:  function MergeSort(array)
2:      if length(array) > 1 then
3:          mid ← ⌊length(array)/2⌋
4:          left ← array[0 : mid]
5:          right ← array[mid : end]
6:          sortedLeft ← MergeSort(left)
7:          sortedRight ← MergeSort(right)
8:          return Merge(sortedLeft, sortedRight)
9:      else
10:         return array
11:     end if
12: end function
13: function Merge(left, right)
14:     result ← empty array
15:     i, j ← 0
16:     while i < length(left)  and  j < length(right) do
17:         if left[i] ≤ right[j] then
18:             append left[i] to result; i ← i + 1
19:         else
20:             append right[j] to result; j ← j + 1
21:         end if
22:     end while
23:     while i < length(left) do
24:         append left[i] to result; i ← i + 1
25:     end while
26:     while j < length(right) do
27:         append right[j] to result; j ← j + 1
28:     end while
29:     return result
30: end function
```

4.2 Quick Sort

Quick Sort is another divide-and-conquer algorithm but works by selecting a pivot element and partitioning the list into subarrays of elements less than and greater than the pivot. It is an in-place algorithm and typically the fastest among comparison-based sorts in practice, with an average-case complexity of $O(n \log n)$. However, its worst-case complexity degrades to $O(n^2)$, especially when sorting already sorted or nearly sorted lists with poorly chosen pivots. Randomizing the pivot selection helps mitigate this. While Quick Sort offers excellent speed and low memory usage, it is unstable, heavily recursive, and can lead to stack overflows on large datasets. These traits make it ideal for general-purpose and fast in-memory sorting but less suitable for highly structured or massive datasets where predictability or stability is needed. [1, 6]

Algorithm 5 QuickSort with In-Place Partitioning

Require: array
Ensure: in-place sorted array

```
1:  procedure QuickSort(array, low, high)
2:      if low < high then
3:          pivotIndex ← Partition(array, low, high)
4:          QuickSort(array, low, pivotIndex - 1)
5:          QuickSort(array, pivotIndex + 1, high)
6:      end if
7:  end procedure
8:
9:  function Partition(array, low, high)
10:     pivot ← array[high]
11:     i ← low - 1
12:     for j ← low to high - 1 do
13:         if array[j] ¡ pivot then
14:             i ← i + 1
15:             Swap(array[i], array[j])
16:         end if
17:     end for
18:     Swap(array[i + 1], array[high])
19:     return i + 1
20: end function
```

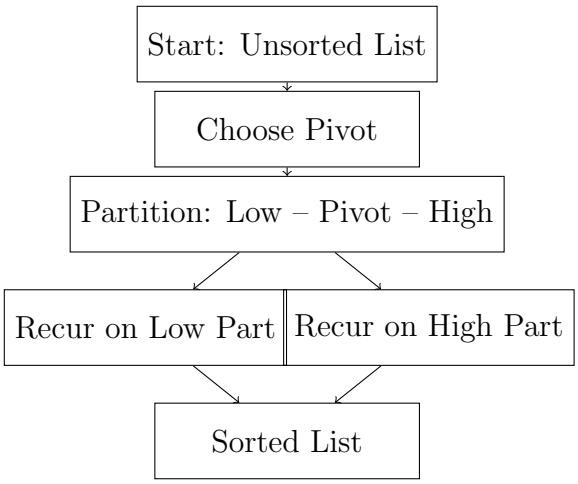


Figure 5: Quick Sort Flowchart

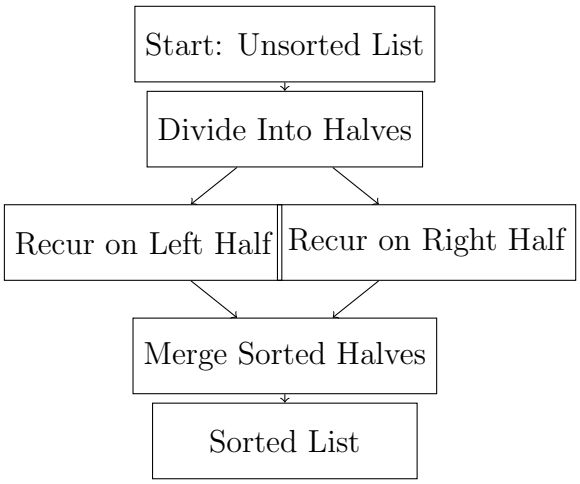


Figure 6: Merge Sort Flowchart

5 Quick-Shuffle

The Quick-Shuffle procedure implements an in-place recursive permutation sampling algorithm inspired by the Quicksort partitioning strategy. Given an array $A[l \ldots r-1]$, the algorithm assigns a uniformly random bit to each element to decide whether it goes to the left or right partition. These bit assignments create a binary split of the array, after which the procedure is recursively applied to each partition. This process continues until all subarrays are of size one or zero, at which point the permutation is complete. Because each element's movement is determined by independent random bits, the procedure ensures that all output permutations are equally likely [2].

Compared to conventional sorting-based approaches, Quick-Shuffle is theoretically more efficient in terms of random-bit usage, as it generates randomness only during element comparisons. In this method, elements are recursively sent to the left or right of a pivot, whose position is determined a posteriori. Once the algorithm terminates, each element receives a unique key, yielding a uniformly random permutation while avoiding unnecessary random-bit consumption.

This algorithms has been implemented in Python as the others, Figure 7 shows that our current implementation of Quick-shuffle remains very crude and inefficient.

Algorithm 6 Quick-Shuffle

Require: array, A of length n
Ensure: shuffled array

function QUICKSHUFFLE(A)
  if $n \leq 1$ **then**
    return A
  end if
  $bits \leftarrow$ `getrandbits`(n)
  $L \leftarrow$ empty list
  $R \leftarrow$ empty list
  for $i \leftarrow 0$ to $n-1$ **do**
    $bit \leftarrow (bits \gg i)\,\&\,1$
    if $bit = 0$ **then**
      append $A[i]$ to L
    else
      append $A[i]$ to R
    end if
  end for
  return QUICKSHUFFLE(L) **concatenated with** QUICKSHUFFLE(R)
end function

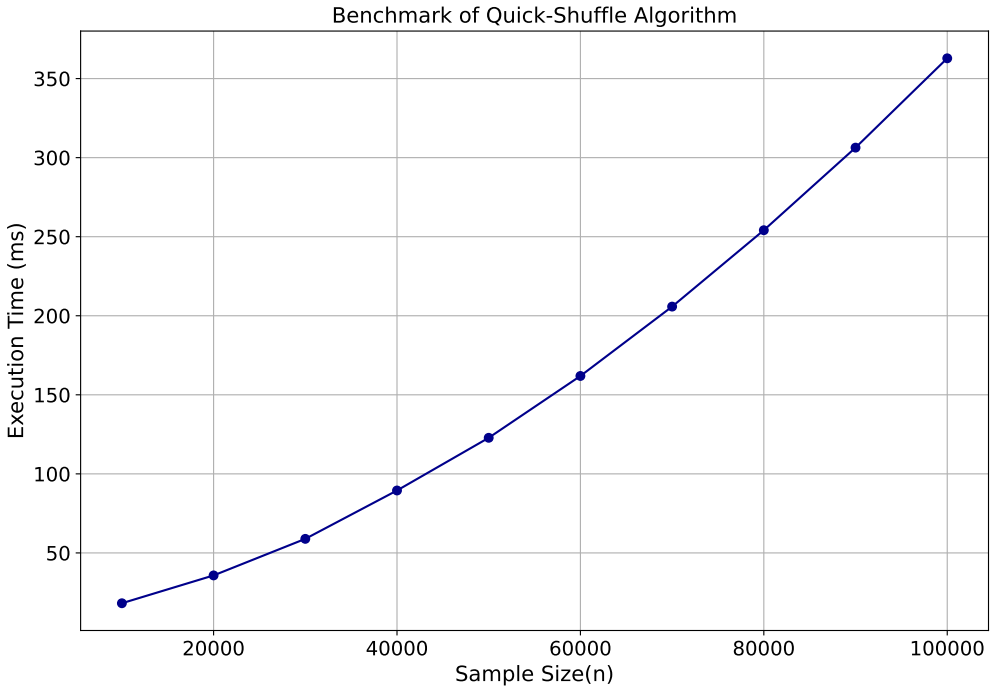


Figure 7: Execution-time of Quick-shuffle

6 Conclusion

In this paper, we have reviewed, analysed, and implemented various approaches to permutation sampling, with an emphasis on efficiency and parallelism. Beyond presenting the review, implementation, and analysis of existing algorithms, we presented an approach based on Randomized Key Sorting, explored in two versions: a uniform-sort-based variant and a multithread-based parallelization. Benchmarking results indicated that the multithread-based Randomized Key Sorting algorithm performed competitively, approaching the efficiency of NumPy's built-in methods. To address random bit efficiency, we also reviewed and implemented two classical sorting algorithms, and subsequently proposed a theoretically efficient method, Quick-Shuffle. However, Quick-Shuffle did not achieve time efficiency in our current implementation. Future research should focus on an efficient implementation of the Quick-Shuffle or developing alternative algorithms that combine both computational speed and random bit efficiency, ensuring scalability for large-scale applications. Given the good performance of the key sorting algorithm, it might be interesting to see how it would scale with a massively multithreaded architecture (e.g. using APIs such as OpenCL, Vulkan or CUDA –Nvidia– and Metal –Apple– depending on the architecture).

References

[1] Khalid Suleiman Al-Kharabsheh, Ibrahim Mahmoud AlTurani, and et al. Review on sorting algorithms a comparative study. International Journal of Computer Science and Security, 2013.

[2] Axel Bacher, Olivier Bodini, Hsien-Kuei Hwang, and Tsung-Hsi Tsai. Generating random permutations by coin tossing: Classical algorithms, new analysis, and modern implementation. ACM Transactions on Algorithms, 2017.

[3] Jens Gustedt. Efficient sampling of random permutations. Journal of Discrete Algorithms, 2008.

[4] I. Mullajonov. Github repository of the project. https://github.com/iscaDev/Permutation-sampling-algorithms, 2025.

[5] NumPy Developers. Numpy documentation. https://numpy.org/doc/stable/index.html, 2025. Accessed: 2025-07-15.

[6] Ramesh Chand Pandey. Study and comparison of various sorting algorithms. Thapar University, Patiala, 2008.